

Software Requirements Document

by Adam Kanoun, Tomas Beatriz Di Assunção, Samy Contuliano Bravo, Naïm Benziani, Chinzorig Adiya, Kerwan Fontaine

Table des matières :

- [1. Introduction](#)
- [2. Architecture du système](#)
 - [2.1. La vue globale du système](#)
 - [2.2. L'architecture client-serveur](#)
- [3. Fonctionnalités du système](#)
 - [3.1. L'authentification](#)
 - [3.2. La gestion des projets](#)
 - [3.3. Importation d'un projet au format interne](#)
 - [3.4. Exportation au format natif \(PNG, BMP,...\) ou format interne](#)
 - [3.5. La création, partage et la suppression de projets](#)
 - [3.6. Les permissions et les rôles à l'intérieur d'un projet](#)
 - [3.7. Le chat](#)
- [4. Edition de la carte](#)
 - [4.1. La navigation et la carte](#)
 - [4.2 Les outils et les couches](#)
 - [4.4. Le remplissage automatique](#)
- [5. Spécification techniques](#)
 - [5.1. Stockage des données](#)
 - [5.2. Gestion de la concurrence des modifications](#)
- [6. Annexes](#)

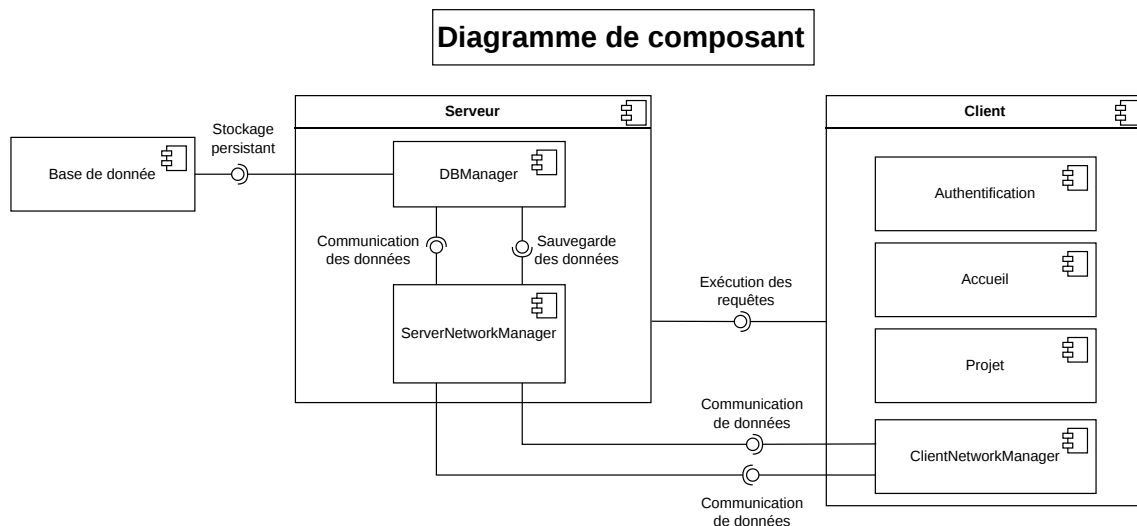
1. Introduction

Ce document est destiné au développement d'une application d'éditeur de cartes RPG, qui est collaborative. L'application permet à plusieurs utilisateurs de travailler ensemble de manière simultanée, en garantissant la cohérence des données et la synchronisation entre ceux-ci.

L'application repose sur une architecture client-serveur. D'une part, il y a le client, qui gère l'interface utilisateur et l'utilisation des fonctionnalités. D'autre part, il y a le système qui gère la centralisation des données et l'envoi des modifications aux autres utilisateurs qui sont connectés au projet. Ce document présente l'architecture globale du système et les principales fonctionnalités accessibles à l'utilisateur.

2. Architecture du système

2.1. La vue globale du système



Le système se base sur une architecture client-serveur, chaque client interagit avec un seul serveur central. Le client va permettre à l'utilisateur de visualiser et modifier une carte via une interface graphique, alors que le serveur gère l'interaction entre les utilisateurs et le stockage des données.

D'abord, les actions réalisées par un utilisateur sont envoyées au serveur via un socket de la bibliothèque "SFML". Ensuite, le serveur va traiter les modifications de l'utilisateur, mettre à jour ces données si nécessaire, et notifie les autres utilisateurs connectés des modifications effectuées pour une synchronisation entre ceux-ci.

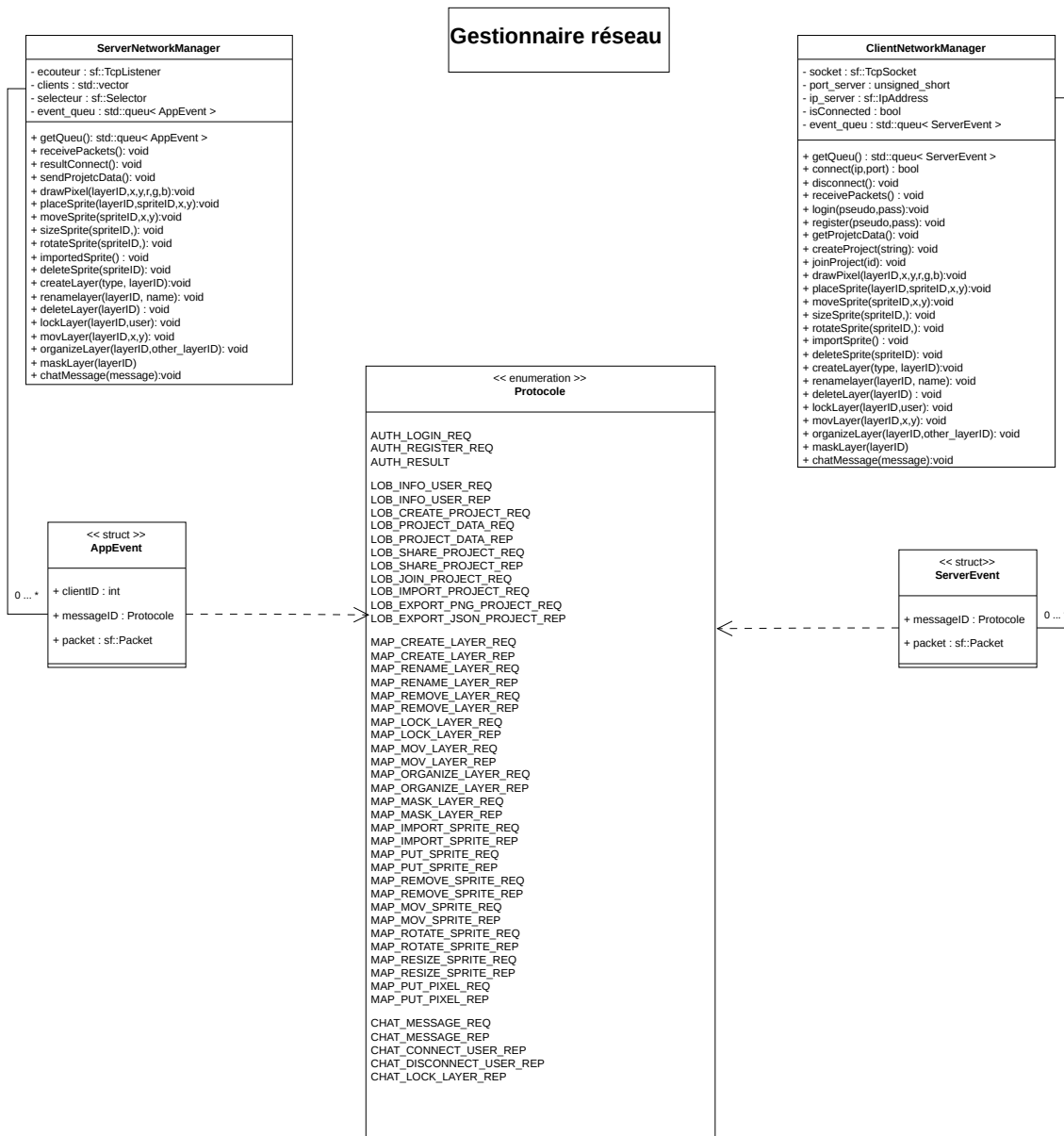
2.2. L'architecture client-serveur

Le client transmet ses commandes au serveur sous forme de requêtes. Le serveur centralise ces actions : il met à jour la base de données et synchronise l'état du projet en informant les autres utilisateurs connectés. Ces échanges sont entièrement asynchrones, ce qui permet de ne jamais bloquer l'application en attendant une réponse du réseau.

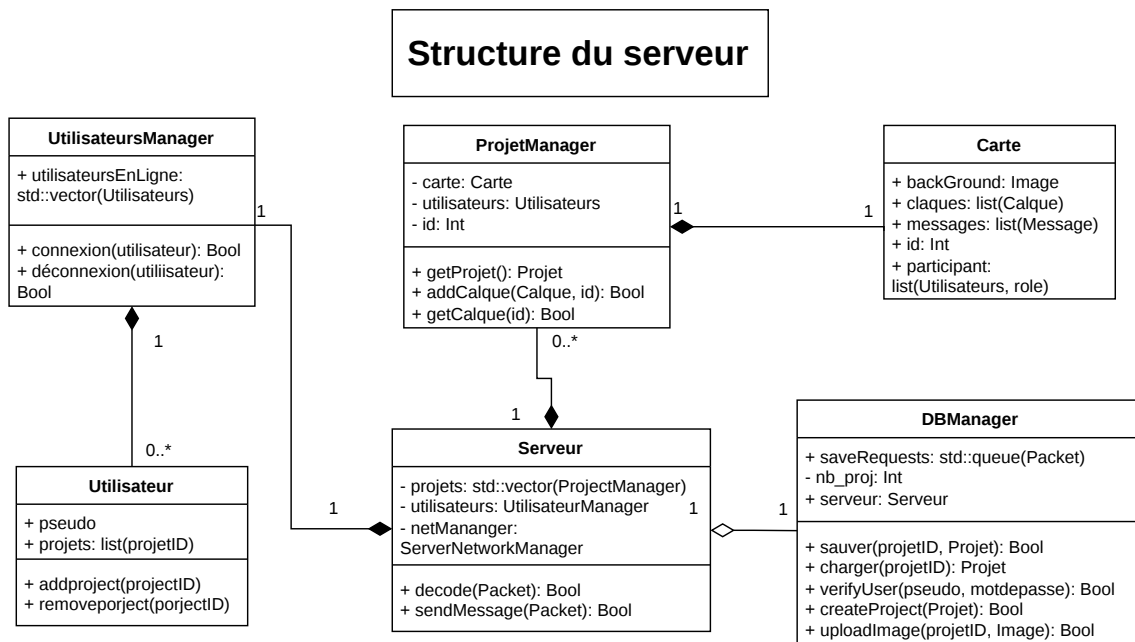
La communication réseau est entièrement encapsulée par deux classes managers. Cette architecture permet de masquer la complexité technique au reste de l'application.

Les deux classes responsables sont :

- **ServerNetworkManager** : Gère le socket d'écoute et maintient la liste des clients connectés. Il centralise la réception des messages et les distribue dans une file d'événements serveur.
- **ClientNetworkManager** : Convertit les actions du jeu en requêtes réseau et, inversement, réceptionne les messages du serveur pour les stocker dans une file d'événements locale. Cette file est ensuite traitée par la boucle principale du jeu pour mettre à jour les modifications.

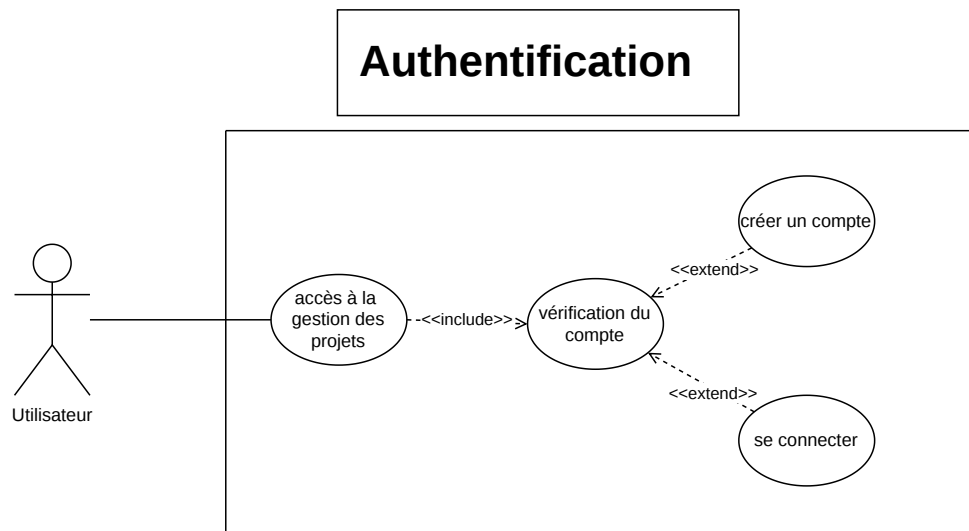


Le nombre important de méthodes dans ces classes est volontaire. Il permet d'exposer chaque fonctionnalité réseau sous forme d'une commande simple, facilitant l'intégration avec le reste du code.

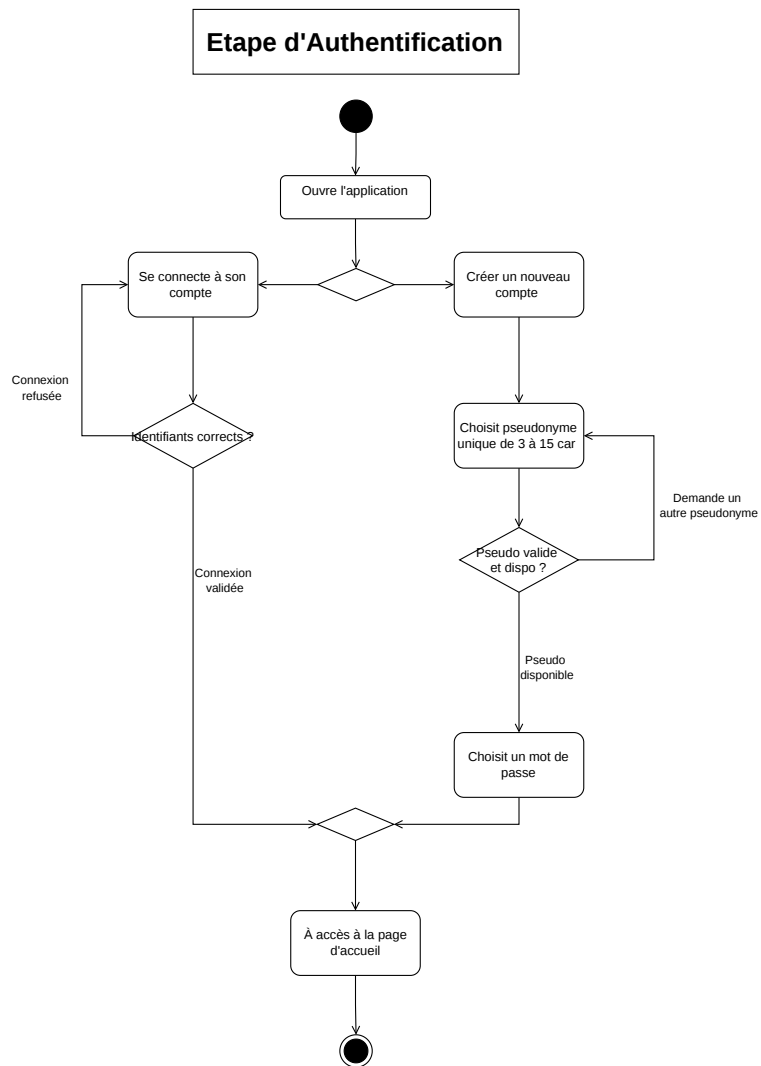


3. Fonctionnalités du système

3.1. L'authentification



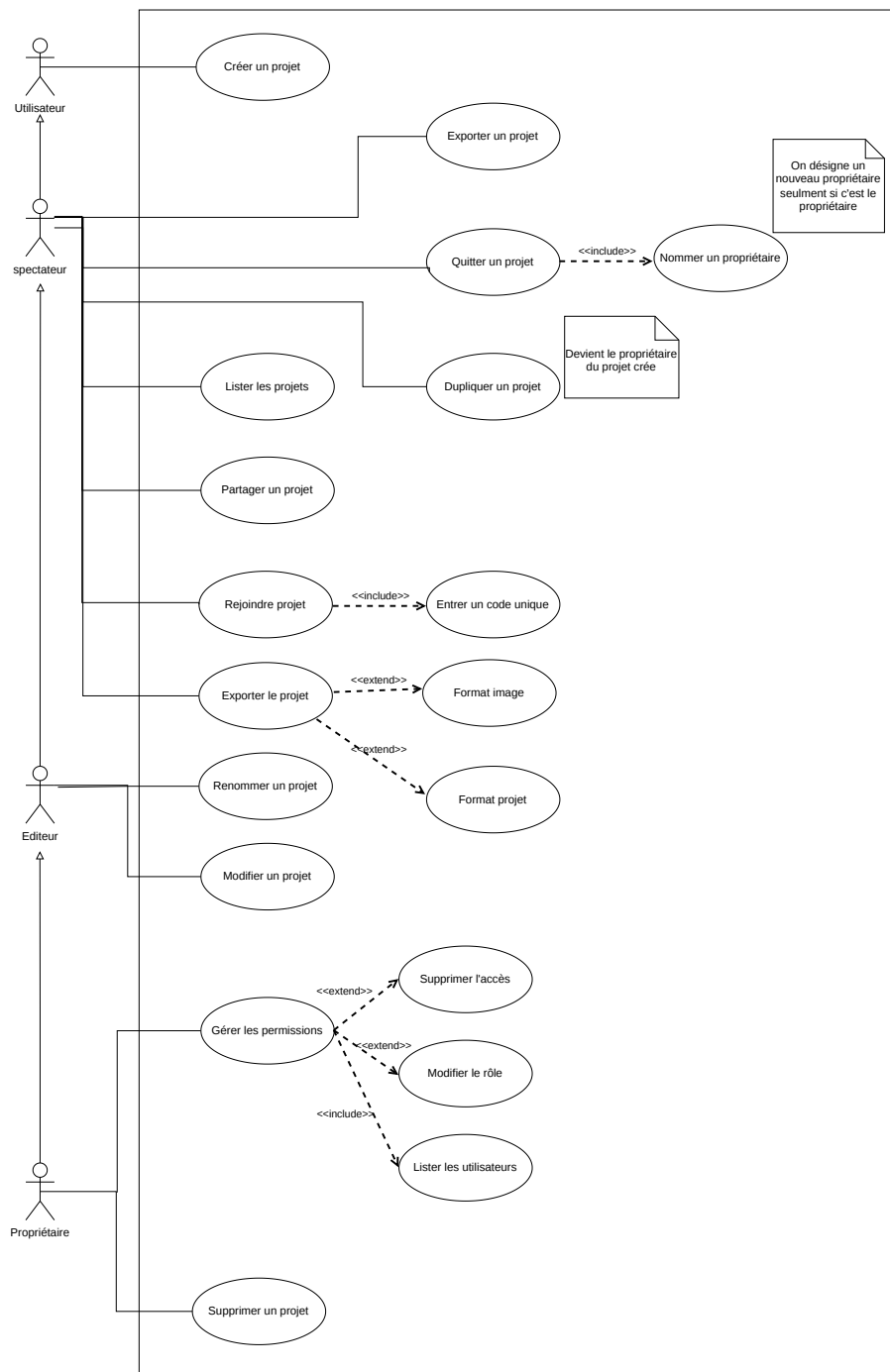
En rentrant dans le projet, l'utilisateur devra passer par une phase d'authentification, c'est ce qu'il lui permettra d'accéder au menu principal qui contient la liste des projets accessibles. Il aura le choix entre se connecter ou s'identifier, s'il choisit la connexion, il devra rentrer un identifiant et un mot de passe valide qui seront vérifiés par le serveur. S'il choisit la création d'un compte, il devra choisir un identifiant non utilisé (entre 3 et 15 caractères) et un mot de passe.

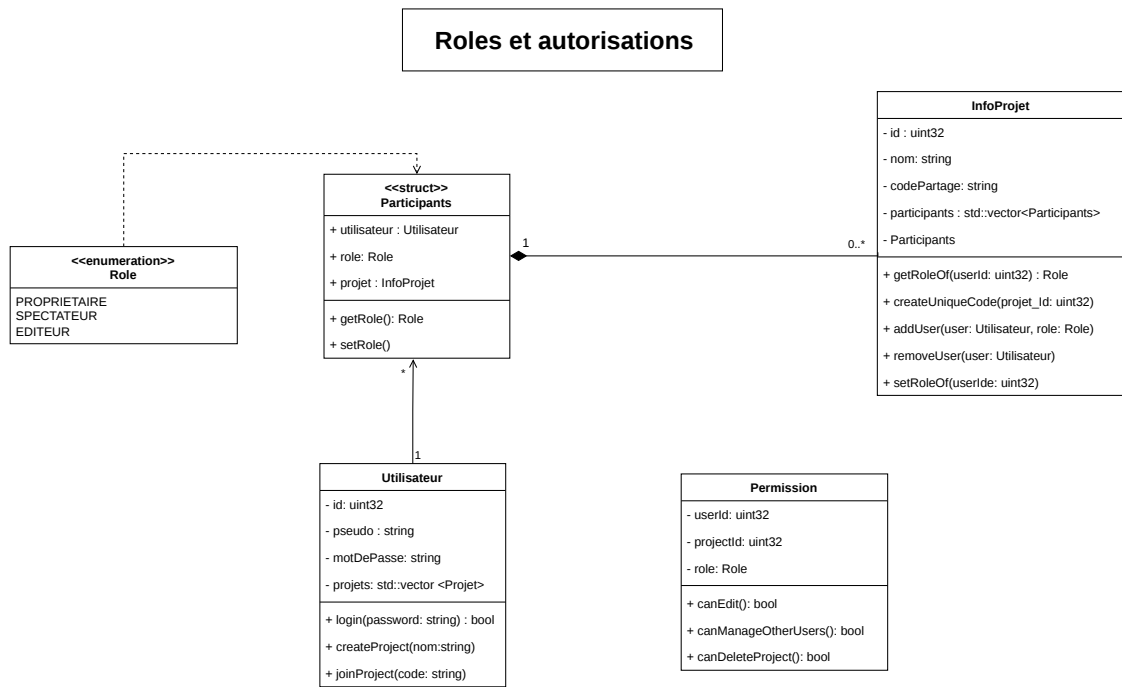


3.2. La gestion des projets

Use case relatif à toutes les actions qu'un utilisateurs peut faire sur les projets auxquels il participe en fonction de son rôle dans chaque projet.

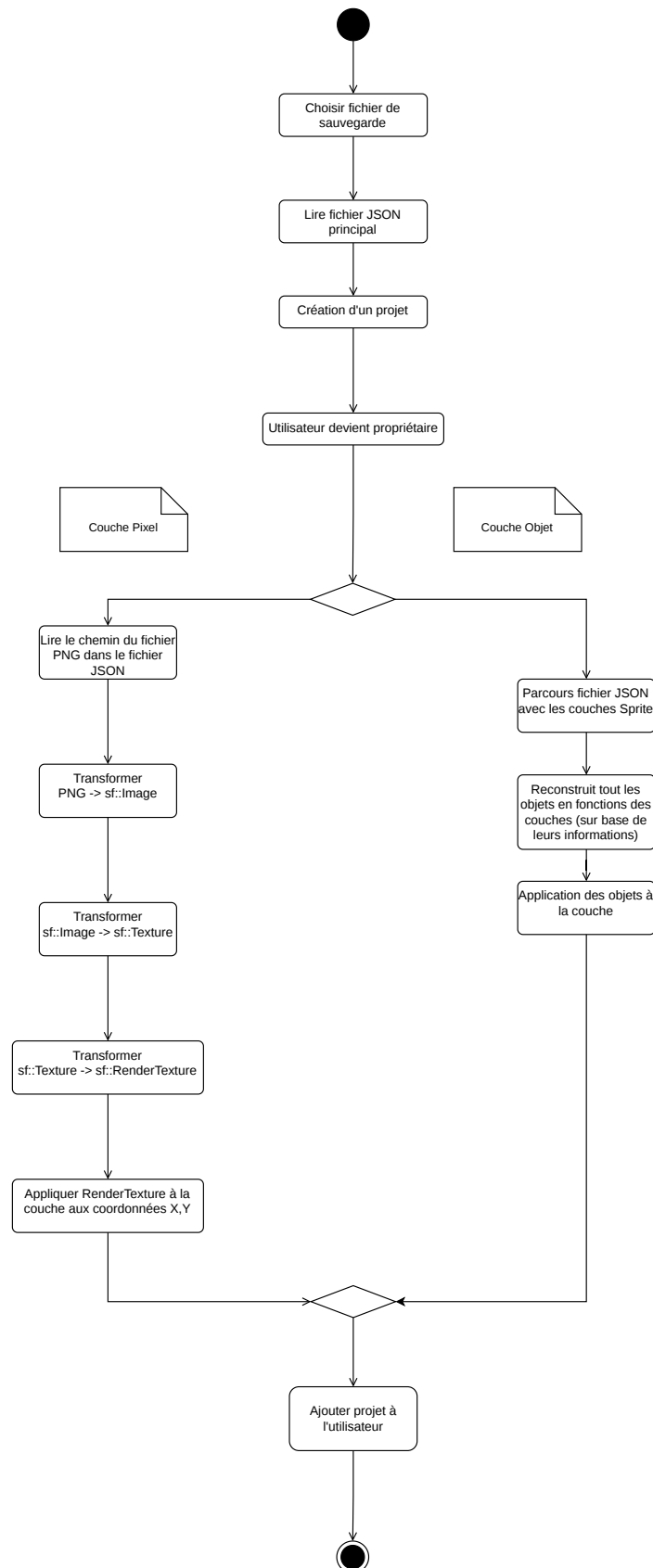
Roles et autorisations



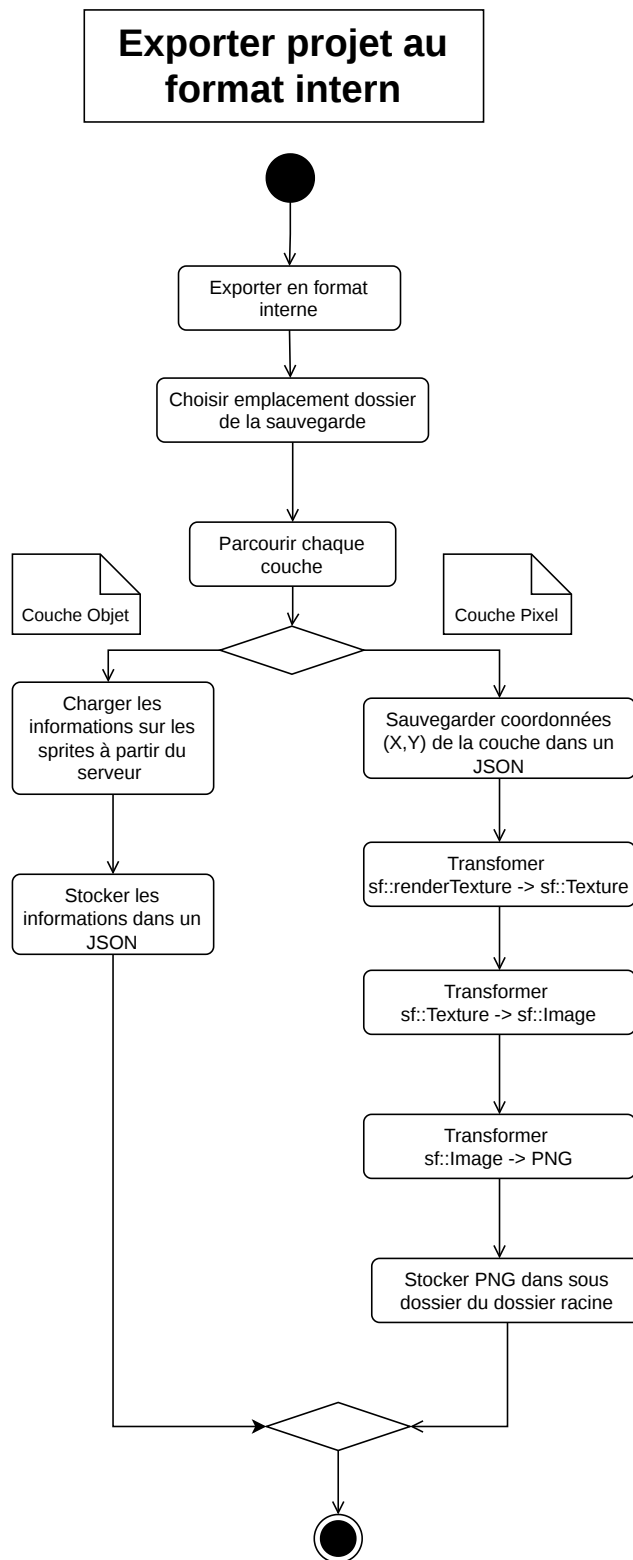


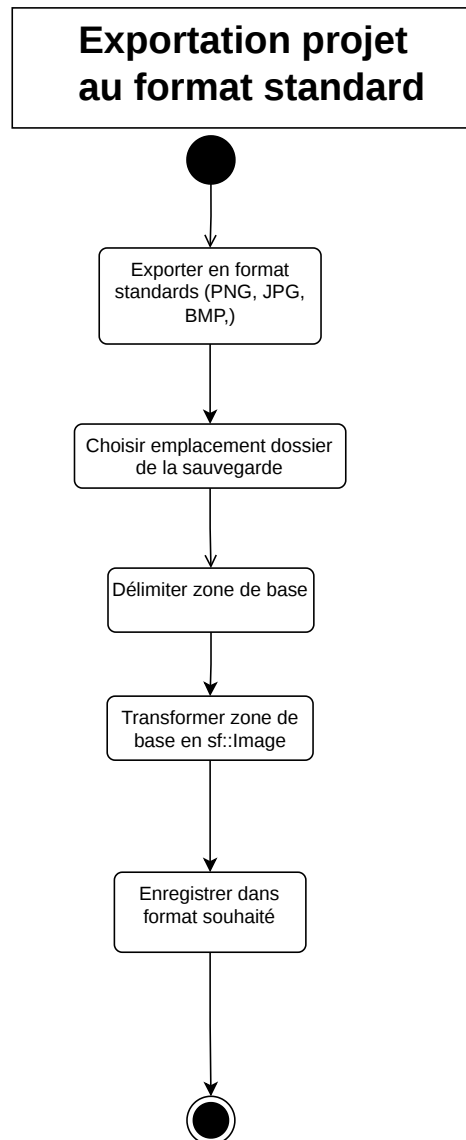
3.3. Importation d'un projet au format interne

Importation d'un projet au format

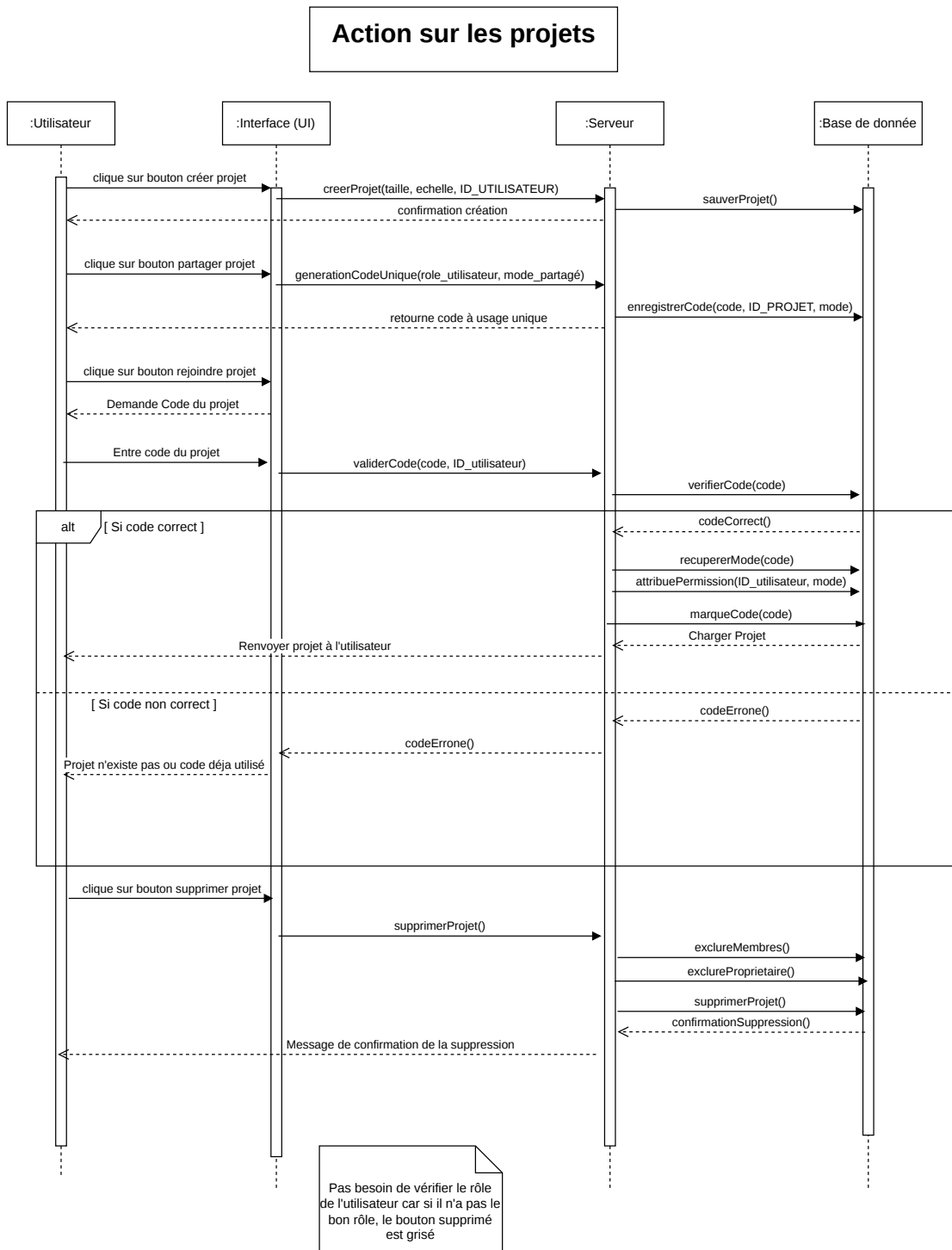


3.4. Exportation au format natif (PNG, BMP,...) ou format interne

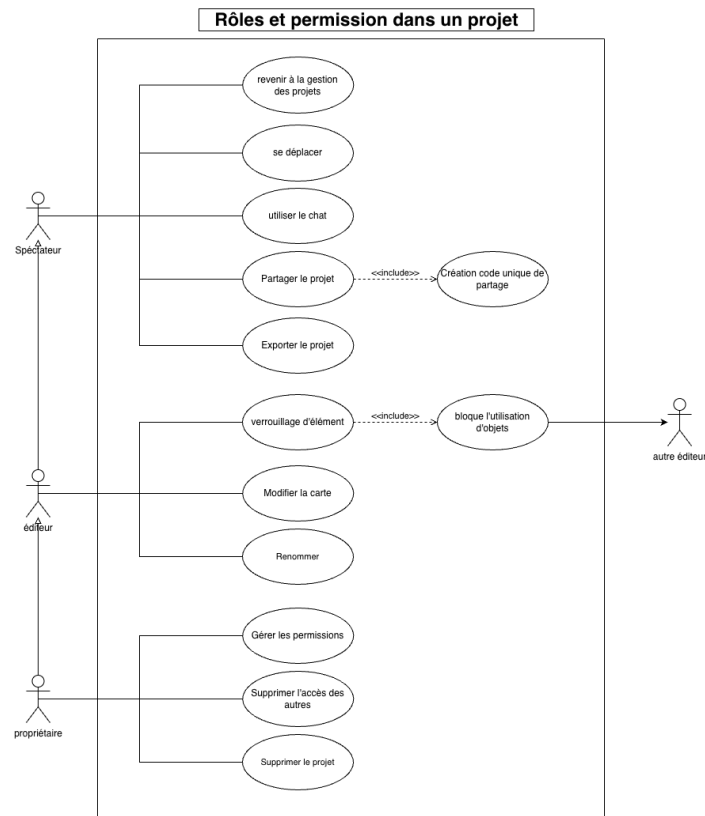




3.5. La création, partage et la suppression de projets



3.6. Les permissions et les rôles à l'intérieur d'un projet



Dans un projet, chaque utilisateur a un rôle spécifique, qui détermine les actions qu'il lui est possible de faire. Propriétaire est le rôle qui contient le plus de droits. Les rôles sont les suivants:

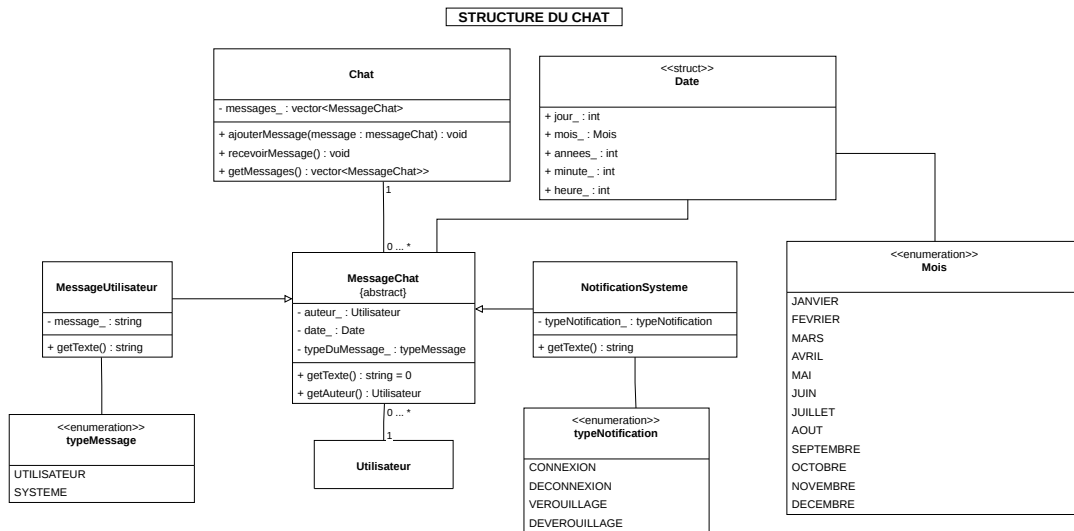
- Propriétaire: il possède tous les droits dans un projet, il peut modifier la carte, supprimer le projet et gérer les rôles des utilisateurs participant au projet.
- Éditeur: peut modifier le contenu de la carte.
- Spectateur: est en lecture seule de la carte.

Tous les rôles ont la possibilité d'utiliser le chat. De plus, les permissions sont gérées par le client, mais aussi par le serveur afin d'éviter des actions non autorisées.

3.7. Le chat

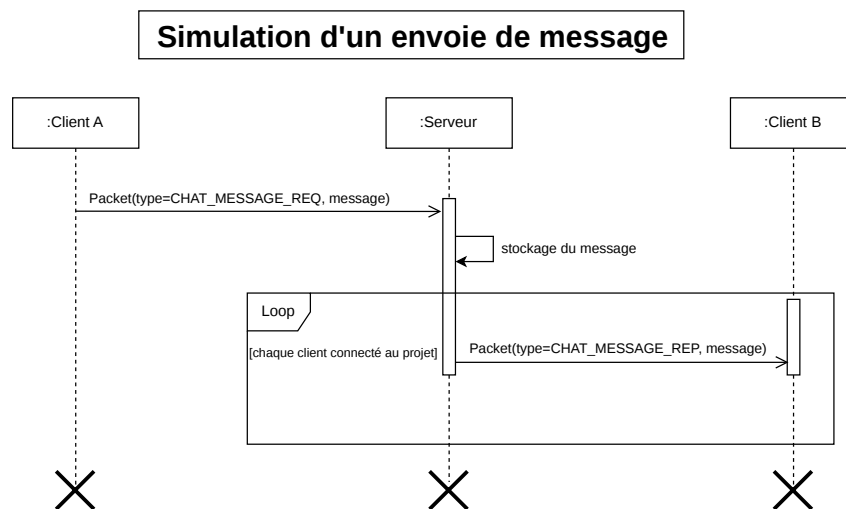
Dans un projet, tous les utilisateurs, qui ont accès au projet, disposent d'un chat commun qui leur permet de discuter. Le chat permet aussi d'afficher les notifications système, comme la connexion ou la déconnexion d'un utilisateur dans le projet, et d'autres actions importantes qui ont été réalisées dans le projet.

Le chat est représenté par une classe qui va stocker l'ensemble des messages échangés sous forme d'une liste. Chaque message est de type `MessageChat`, ce qui permet de distinguer les messages utilisateurs des notifications système. Chaque message contient le contenu du message, l'auteur et l'heure à laquelle elle a été envoyée. Pour une notification système, elle est caractérisée par un type.



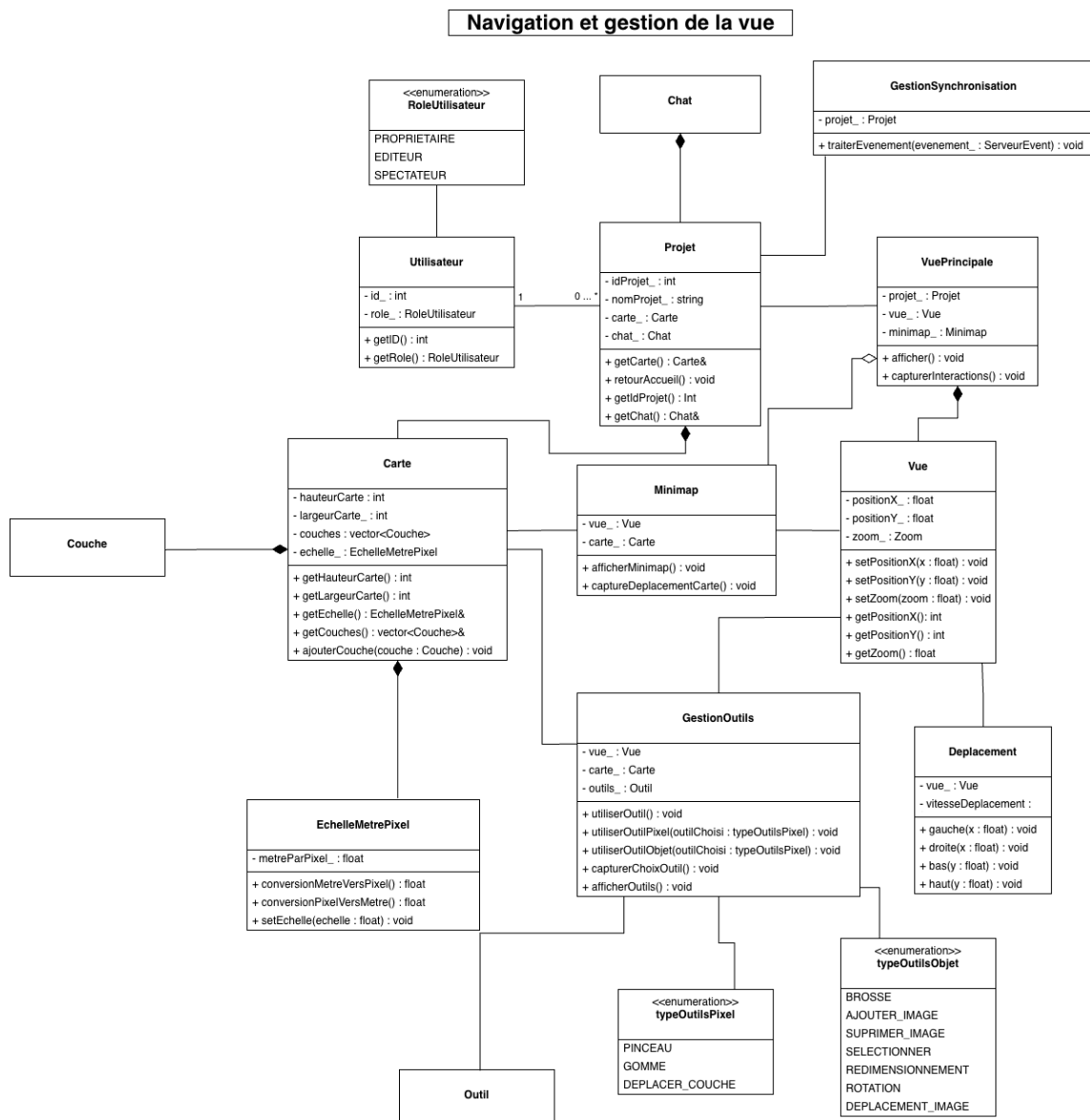
Quand un utilisateur veut envoyer un message dans le chat, celui-ci est transmis au serveur sous la forme d'un paquet réseau. Le serveur va recevoir le message, le sauvegarder pour conserver un historique complet du chat, puis l'envoie à l'ensemble des utilisateurs connectés au projet.

L'envoi des messages est fait de manière asynchrone, afin d'éviter de bloquer le bon fonctionnement du système. Du côté client, les messages reçus sont ajoutés au chat et affichés par le client.



4. Edition de la carte

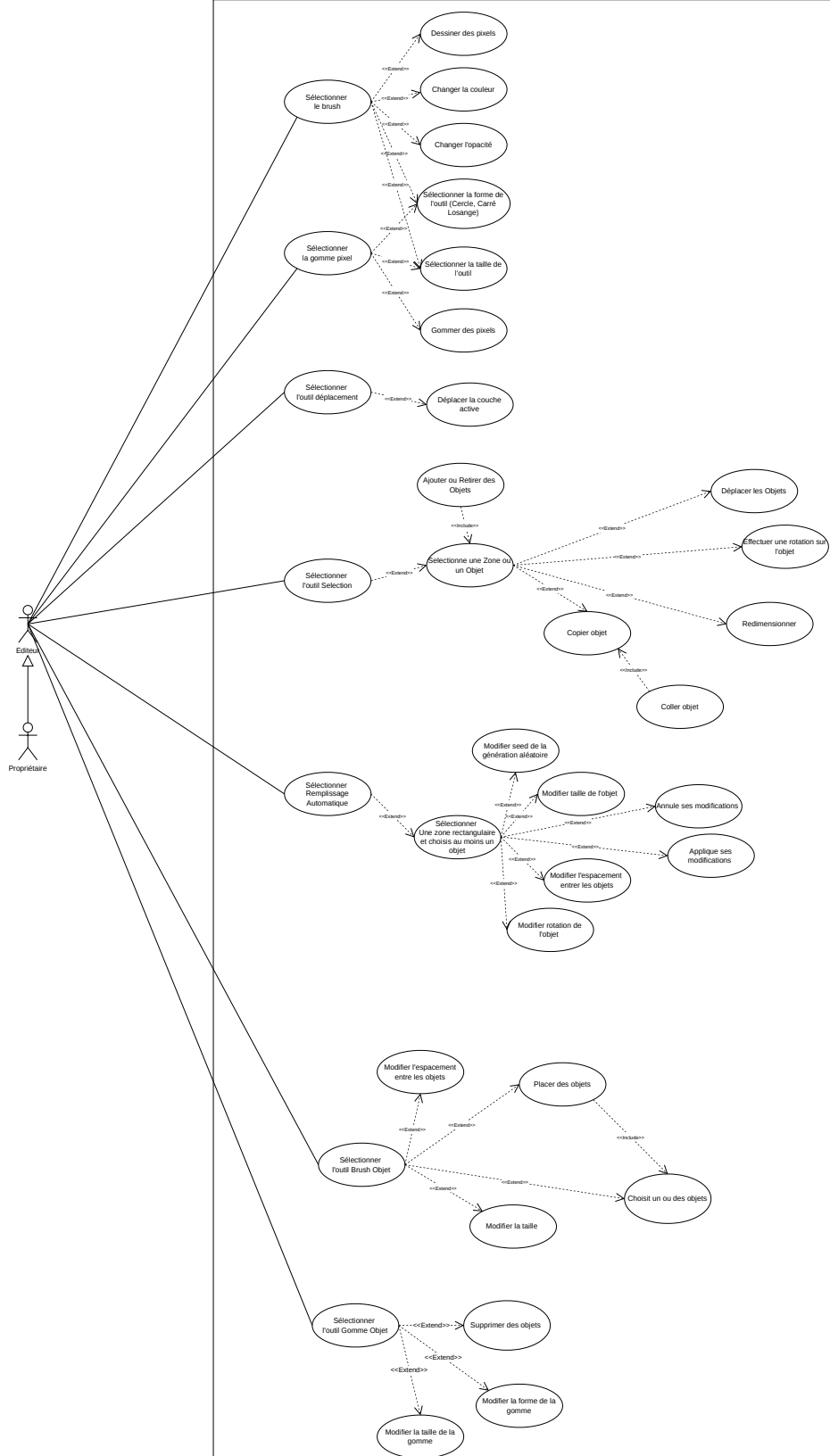
4.1. La navigation et la carte



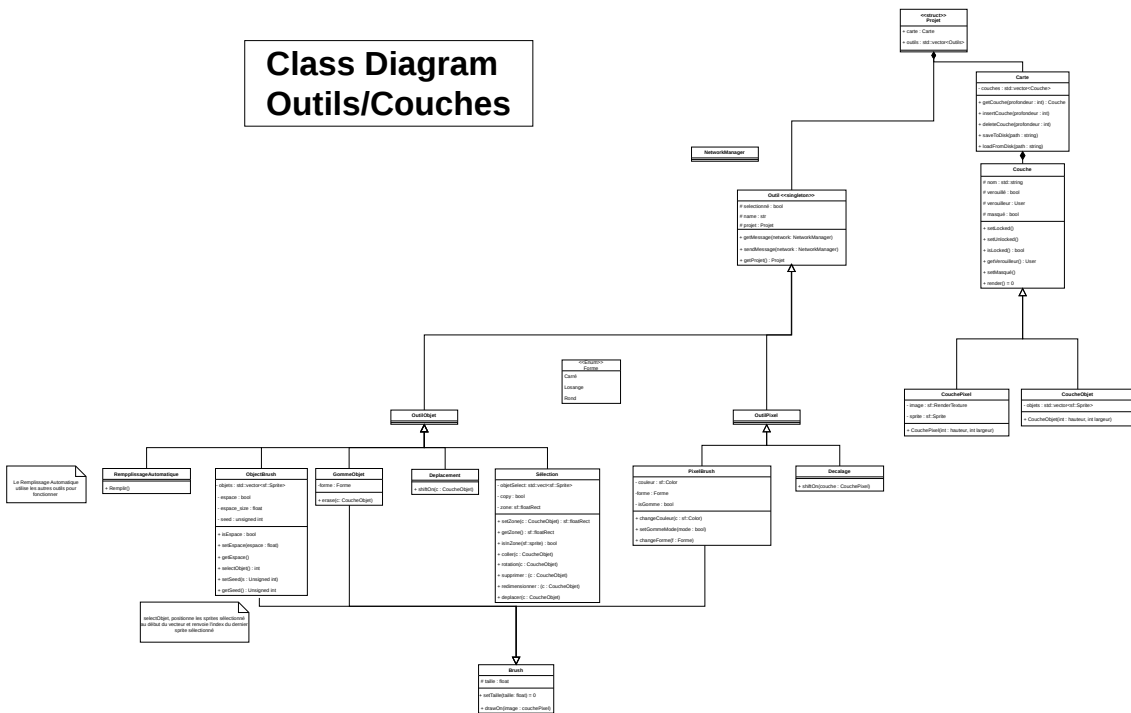
L'utilisateur a la possibilité de naviguer, en utilisant les touches "ZQSD" du clavier, par la minicarte ou par les flèches directionnels du clavier.

4.2 Les outils et les couches

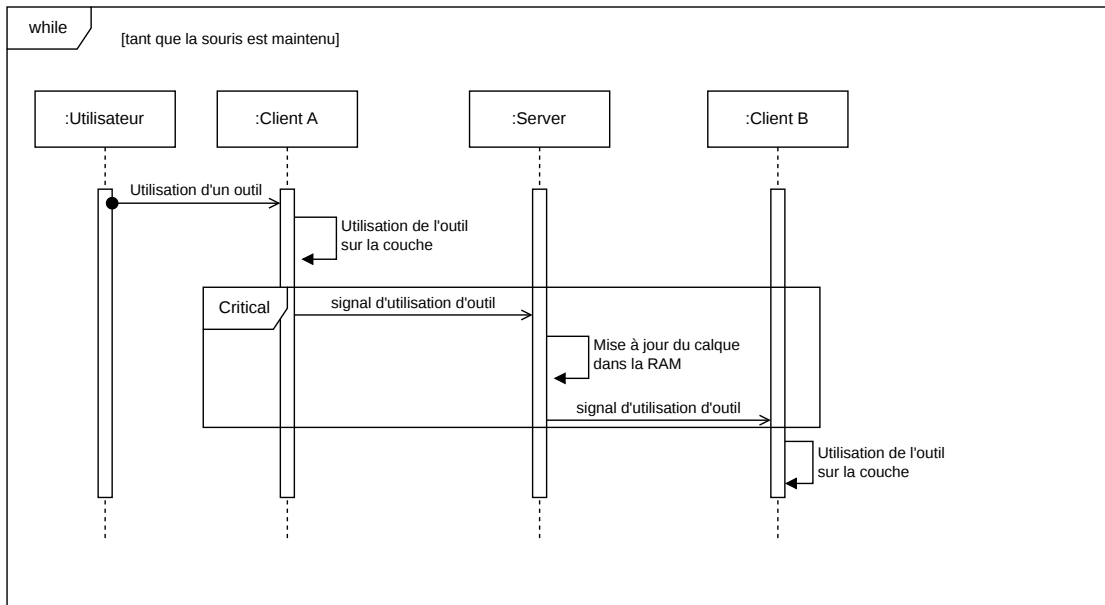
Use Case Outils



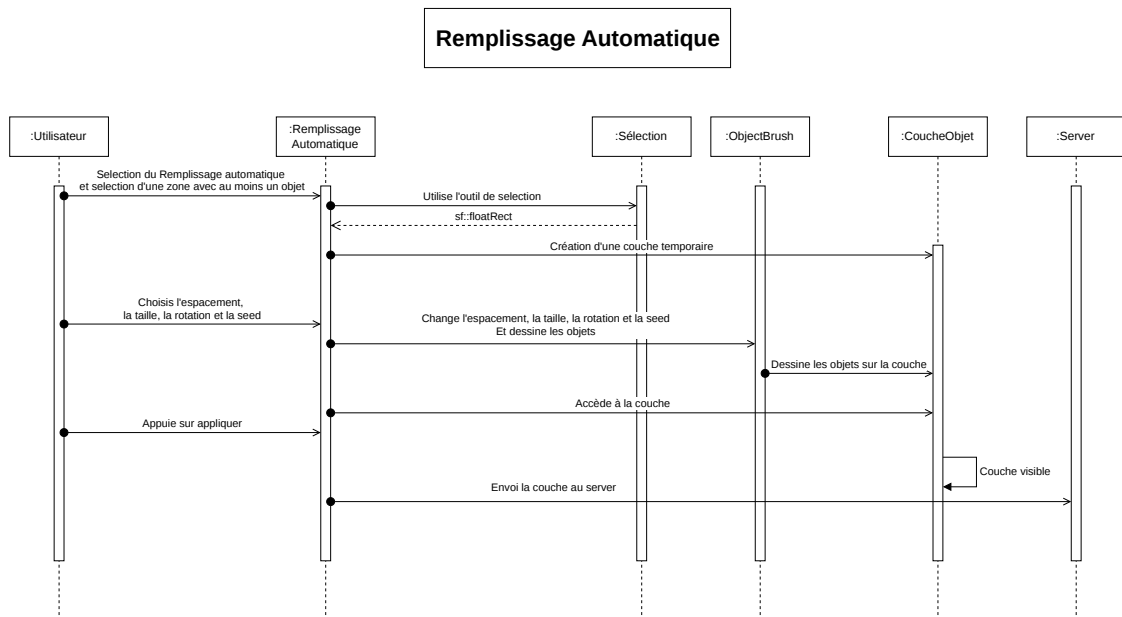
Class Diagram
Outils/Couches



Utilisation Outils



4.4. Le remplissage automatique



5. Spécification techniques

5.1. Stockage des données

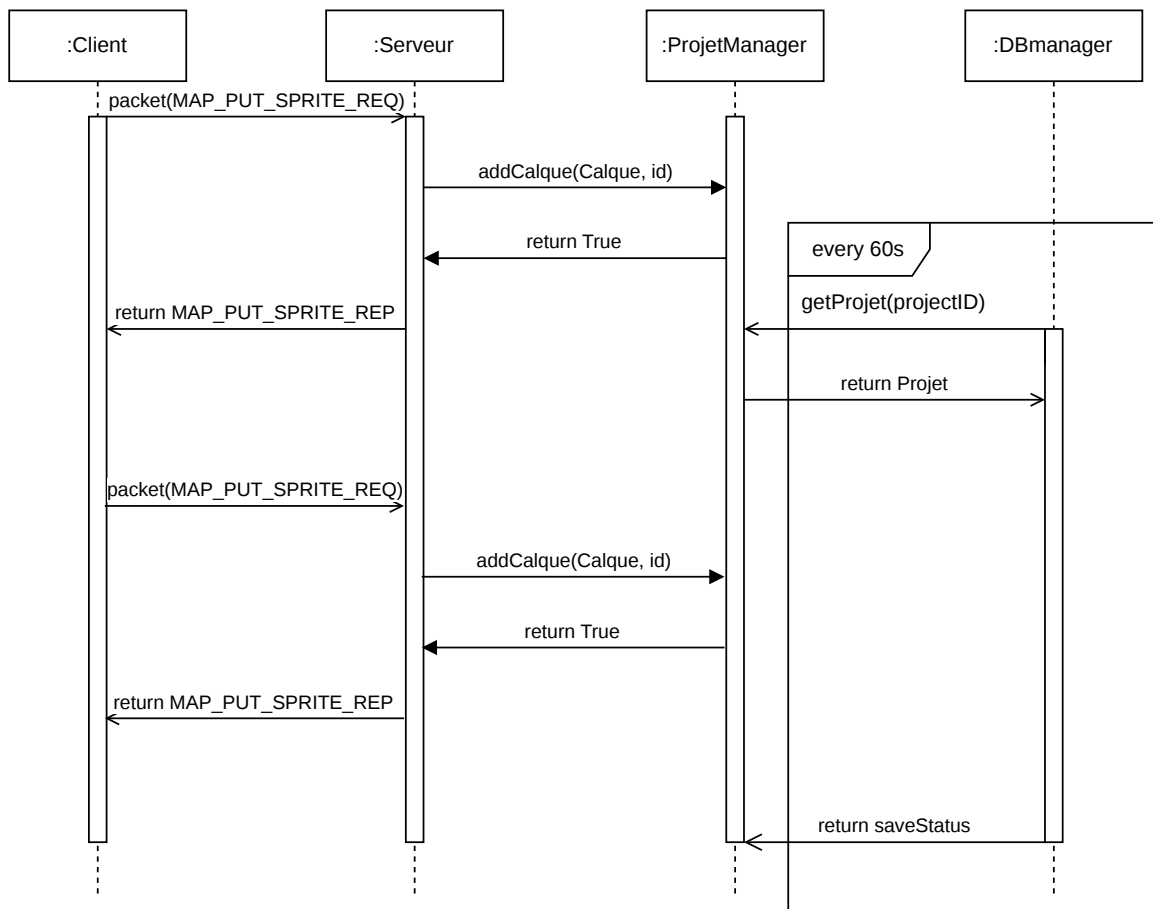
Le stockage des données se fera directement par fichier, sans faire appel à un Système de Gestion de Base de Données (SGBD).

En effet, la partie "image" projet peut être décomposé en un dossier contenant les médias image qui ont été importés dans le projet, un dossier sprite, contenant un encodage du contenu des différents calques pixels et un fichier JSON qui donne la structure des différents calques (position, agrandissement, niveau, nom dans les fichiers de l'image à charger, etc). La partie "communautaire" (c'est-à-dire le chat, les participants et leurs rôles) est à part, encodée dans 2 fichier: l'un contenant la liste des participants avec leur rôles et un autre contenant l'entièreté du chat.

Ce découpage rend le maintien des données et leur modifications conceptuellement assez simple: importer une image ajoute un fichier dans le dossier image, de même pour l'ajout de pixel tandis que la modification de paramètre de calque modifie un objet mémoire qui contient la liste des calques du projet. Les sauvegardes ponctuelles ne correspondent qu'à une conversion de l'objet "projet" en RAM en un fichier JSON, les images et le contenu des calques pixels étant à priori sauvegardés sur le disque dur lors de leur envoi au serveur.

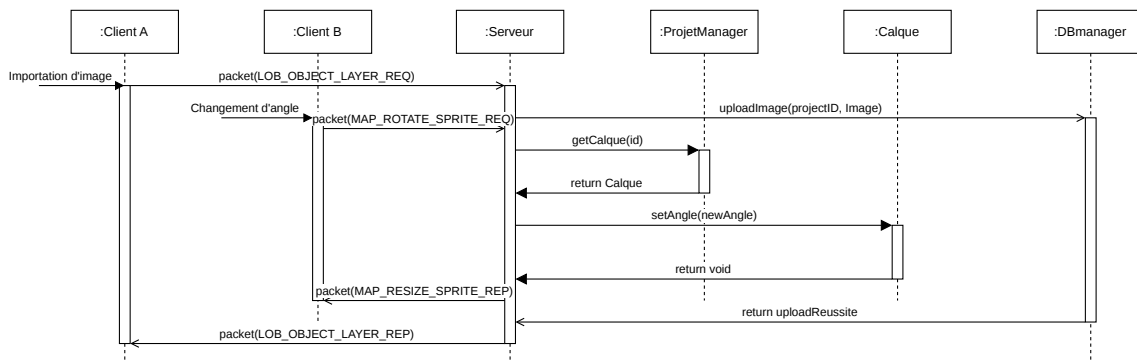
Le format JSON est actuellement envisagé car plus simple à débayer, son contenu étant intelligible pour l'humain. Une piste futur d'amélioration envisagée est de remplacer le JSON par un format propre à nous qui est la traduction en binaire de l'objet de classe projet (en déréférencant les pointeurs de sa liste). Ainsi, les écritures en mémoire seraient plus rapides.

Sauvegarde automatique en arrière plan du projet



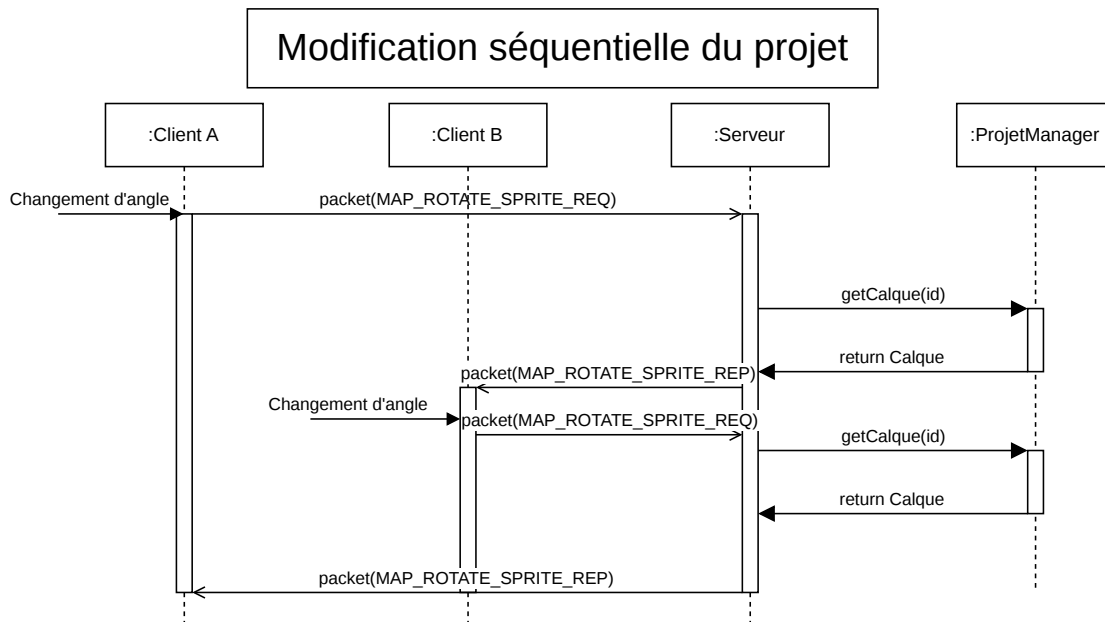
Pour les sauvegardes régulières, le serveur fait appel à un programme qui tourne dans un thread à l'exception qu'il ne s'occupe exclusivement que des demandes liées à la lecture ou le chargement de données depuis le disque dur. Le faire tourner dans un thread séparé nous assure que celui-ci est asynchrone au serveur. Ainsi nous sommes sûrs qu'une demande de sauvegarde ne figera pas l'application à cause du "long" temps d'écriture.

Modification de projet concurrente à une importation d'image



5.2. Gestion de la concurrence des modifications

Il nous suffit de gérer les requêtes de modification dans l'ordre de leur arrivée et des les placer dans un structure de file (premier rentré, premier sortie).



6. Annexes

Pour ce projet, nous utilisons la bibliothèque SFML. Elle nous permettra d'avoir un rendu graphique, capturer les interactions de l'utilisateur par sa souris ou clavier, ainsi que l'envoi des données vers le serveur et inversement, etc.